

Laboratorio Práctico 02: Fundamentos, Estructuras y Control de Flujo

Curso:	Python para Análisis de Datos (ETL)	Instructor:	Christian Condori
--------	-------------------------------------	-------------	-------------------

Instrucciones Generales

- Desarrolle las soluciones de este laboratorio directamente en celdas de código en su entorno de **Google Colab**.
- Asegúrese de comentar su código explicando la lógica detrás de cada paso.
- No utilice librerías externas (como Pandas) para este laboratorio. El objetivo es evaluar su dominio de Python nativo.

Caso de Negocio: Limpieza y Clasificación de Transacciones

Usted forma parte del equipo de Ingeniería de Datos de una pasarela de pagos. El sistema heredado (legacy) envía los registros de las transacciones diarias en un formato de texto crudo y desordenado. Su tarea es crear un script que reciba este registro, lo limpie, evalúe el nivel de riesgo de la transacción y consolide la información en un formato estructurado para el reporte de cierre.

Parte 1: Ingesta y Limpieza de Cadenas (Strings)

El sistema ha arrojado la siguiente cadena de texto correspondiente a la última transacción. Cópiala exactamente igual en su Colab:

```
log_crudo = "$$ [ESTADO:ok] || trx_id:9934 || monto:S/.1250.50 || alerta:falso$$ "
```

Misiones:

1. Utilice el método adecuado para eliminar los espacios en blanco y los símbolos de dólar \$\$ que se encuentran en los extremos de la cadena.
2. Convierta toda la cadena resultante a letras mayúsculas.
3. El delimitador que separa cada campo es la doble barra vertical ||. Utilice el método correcto para dividir la cadena en partes usando este delimitador.
4. Guarde el resultado final en una variable llamada `lista_campos` e imprímala en pantalla usando `print()`.

Parte 2: Colecciones y Extracción Analítica

Ahora que tiene una lista de elementos separados, necesitamos aislar el valor numérico para poder operar con él.

Misiones:

1. Acceda al tercer elemento de su `lista_campos` (recuerde cómo funcionan los índices en Python) donde se encuentra la información del monto.
2. Utilizando técnicas de *Slicing* (rebanado) o métodos de strings, extraiga únicamente la porción numérica (es decir, excluya el texto "MONTO:S/.").
3. Convierta ese texto extraído a un tipo de dato decimal (float) y guárdelo en una variable llamada `monto_transaccion`.

Parte 3: Lógica Condicional y Tuplas de Control

El departamento de Prevención de Fraude tiene umbrales estrictos para auditar transacciones. Estos umbrales no deben ser modificados por error durante la ejecución del programa.

Misiones:

1. Cree una **Tupla** llamada `umbrales_auditoria` que contenga dos valores: (500.0, 2000.0). El primer valor es el límite de revisión manual, y el segundo es el límite de bloqueo por presunto fraude.
2. Implemente una estructura condicional (if - elif - else) que evalúe la variable `monto_transaccion`:
 - Si el monto es estrictamente mayor al límite de bloqueo (posición 1 de la tupla), imprima: *"ALERTA ROJA: Transacción bloqueada. Supera los \$2000."*
 - Si el monto es mayor al límite de revisión (posición 0 de la tupla) pero menor o

igual al de bloqueo, imprima: *"ALERTA AMARILLA: Transacción enviada a revisión manual."*

- Si no se cumple ninguna de las anteriores, imprima: *"VERDE: Transacción procesada automáticamente."*

Parte 4: Diccionarios y Automatización con Bucles

Para enviar la información al sistema de reportes, debemos consolidarla en un diccionario e iterar sobre los resultados.

Misiones:

1. Cree un **Diccionario** llamado `reporte_diario` con las siguientes llaves y valores:
 - `"fecha"`: "2026-06-07"
 - `"lotes_procesados"`: Una lista con los montos [350.0, 1250.50, 90.0, 2100.0]
 - `"operador"`: Ingrese su Nombre y Apellido.
2. Utilice un bucle `for` combinado con la función `enumerate()` para recorrer la lista de `lotes_procesados` dentro del diccionario.
3. Dentro del bucle, utilice una *f-string* para imprimir: *"Lote N° [Índice] procesado por un valor de \$[Monto]"*. Haga que el índice comience en 1.
4. Simule un proceso de conexión de red inestable utilizando un bucle `while`. Configure un contador de intentos inicializado en 1. El bucle debe imprimir *"Intentando conectar al servidor de destino... Intento [X]"* y detenerse automáticamente al llegar al intento número 3.

Entregable

Una vez completados todos los pasos en su Google Colab, descargue el notebook en formato `.ipynb` y súbalo a la plataforma del curso en la tarea correspondiente a la Clase 2.